

GAMMA · SECURITY REVIEW

Gamma Position Managers.

A security review of Position Managers.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
Gamma	Position Managers	Guardian	June 9 – July 7, 2025	July 14, 2025

§ 00

Contents.

76 pages — overview, confidence ranking, scope, 58 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 49 findings	P. 11
	Remediation Review · 9 findings	P. 65
§ 05	Disclaimer	P. 75
§ 06	About Guardian	P. 76

§ 01

Overview.

Guardian reviewed Position Managers across 2 review rounds. The final report includes 5 High, 10 Medium, 32 Low, 11 Informational findings.

AUDIT FIRM	REPOSITORY
Guardian	GammaStrategies/HypervisorNFPM, GammaStrategies/MultiPositionManager
CLIENT	METHODOLOGY
Gamma	Smart contract · Manual · Main Review · Remediation Review
PROJECT	MAIN REVIEW COMMIT
Position Managers	4fa8879de218
PREPARED BY	
Guardian	
AUDIT WINDOW	
June 9 – July 7, 2025	
FINAL REPORT	
July 14, 2025	

1.1 Confidence Ranking

3 / 5

Moderate
Confidence

Guardian assigns a Confidence Ranking of 3 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
Critical	0	0	0	0	0
High	5	0	3	0	2
Medium	10	0	3	0	7
Low	32	0	21	0	11
Informational	11	0	8	0	3
All severities	58	0	35	0	23

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence ASSIGNED	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed Position Managers in `GammaStrategies/HypervisorNFPM`, `GammaStrategies/MultiPositionManager`. The scope includes 2,767 source lines across 11 files.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH

Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM

Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW

Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH

Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM

Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW

Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
contracts/UniProxyETH.sol	181	290
contracts/MultiFeeDistribution.sol	301	525
contracts/HypervisorNFPM.sol	483	743
contracts/ClearingV3NFPM.sol	177	273
contracts/libraries/RewardCalculations.sol	119	157
contracts/libraries/PositionValue.sol	75	96
contracts/libraries/PositionManagementLibrary.sol	192	228
src/UniProxyV2.sol	100	152
src/PoolManagerUtils.sol	417	485
src/MultiPositionManager.sol	598	778
src/ClearingV3.sol	124	164

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
H-01	No Share Slippage Protection In deposit	Frontrunning	HIGH	RESOLVED	p. 11
H-02	Wrong Usage Of collectRewards Return Values	Logical Error	HIGH	RESOLVED	p. 13
M-01	Unfortunate Withdraw Timing Can Lead To Loss	MEV	MEDIUM	RESOLVED	p. 15
M-02	Fee Checked Incorrectly	Logical Error	MEDIUM	RESOLVED	p. 16
M-03	Tick Bounds Can Be Miscalculated	Logical Error	MEDIUM	RESOLVED	p. 17
M-04	Overflow In Price Calculation Not Fixed	Math	MEDIUM	RESOLVED	p. 18
M-05	Algebra Oracle Vulnerable To TWAP Lookback DoS	DoS	MEDIUM	ACKNOWLEDGED	p. 19
M-06	Rebalance Can Revert During Emergency Mode	DoS	MEDIUM	RESOLVED	p. 21
M-07	Clearing Reverts On Inactive Limit Positions	DoS	MEDIUM	RESOLVED	p. 22
M-08	decreaseLiquidity Doesn't Collect Funds	Unexpected Behavior	MEDIUM	ACKNOWLEDGED	p. 23
L-01	Excess Native Refunds May Revert For Contracts	Best Practices	LOW	ACKNOWLEDGED	p. 24
L-02	FeeRecipient Update Can Be Frontrun	Documentation	LOW	ACKNOWLEDGED	p. 25
L-03	Unnecessary Approvals	Unexpected Behavior	LOW	RESOLVED	p. 26

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-04	Unused ReentrancyGuard In ClearingV3.sol	Informational	LOW	RESOLVED	p. 27
L-05	Incorrect Error Usage	Informational	LOW	RESOLVED	p. 28
L-06	Incorrect Price For Negative Tick	Logical Error	LOW	ACKNOWLEDGED	p. 29
L-07	mintLiquidity Mishandles Fees	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 30
L-08	getTotalAmountsPlusFees May Be Stale	Warning	LOW	ACKNOWLEDGED	p. 31
L-09	Possible 0 Token Transfer	Rewards	LOW	RESOLVED	p. 32
L-10	setNftIds Doesn't Collect Funds	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 33
L-11	Unnecessary Check In addReward()	Best Practices	LOW	ACKNOWLEDGED	p. 34
L-12	simulateOneSecondGrowth Is Not Used	Best Practices	LOW	ACKNOWLEDGED	p. 35
L-13	Slippage Not Enforced For Limit Positions	Frontrunning	LOW	ACKNOWLEDGED	p. 36
L-14	Unnecessary Assignment	Best Practices	LOW	ACKNOWLEDGED	p. 38
L-15	Staking Token Can Be Set As Reward Token	Best Practices	LOW	RESOLVED	p. 39
L-16	Clearing Doesn't Consider Liquidity	Informational	LOW	ACKNOWLEDGED	p. 40
L-17	CEI Pattern Not Followed	Best Practices	LOW	ACKNOWLEDGED	p. 41
L-18	Withdrawals Can Be Weaponized	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 42
L-19	Potential 0 Transfer On Withdraw	Validation	LOW	RESOLVED	p. 43
L-20	ETH Surplus May Not Be Refunded	Logical Error	LOW	RESOLVED	p. 44

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-21	inMin Is Not Enforced When liquidity = 0	Validation	LOW	ACKNOWLEDGED	p. 46
L-22	UniProxy.transferETH() Is Permissionless	Informational	LOW	ACKNOWLEDGED	p. 47
L-23	Whitelisted Depositors Are Not Exempted	Validation	LOW	RESOLVED	p. 48
L-24	UniProxyETH Is Vulnerable To Tokens With Hooks	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 49
L-25	Zero-Interval TWAP Always Returns Zero	Logical Error	LOW	RESOLVED	p. 50
L-26	Misaligned Limit Positions Due To slot0.tick	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 51
L-27	Inconsistent Price Threshold Check	Math	LOW	ACKNOWLEDGED	p. 52
L-28	Lack Of Checkpointing When Setting New Fee	Logical Error	LOW	RESOLVED	p. 53
L-29	mintLiquidity Causes Accounting Mismatch	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 54
L-30	Zero Shares Received During Deposit	Logical Error	LOW	RESOLVED	p. 55
I-01	Incorrect Comment For removeRewardToken	Documentation	INFORMATIONAL	RESOLVED	p. 56
I-02	Incorrect Comment For Managers	Documentation	INFORMATIONAL	ACKNOWLEDGED	p. 57
I-03	Missing Zero Address Check In Constructor	Validation	INFORMATIONAL	RESOLVED	p. 58
I-04	Use Require Over Assert	Best Practices	INFORMATIONAL	RESOLVED	p. 59
I-05	Redundant Fee Initialization	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 60
I-06	Liquidity May Not Fit Into uint128	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 61
I-07	Outdated Comments	Documentation	INFORMATIONAL	ACKNOWLEDGED	p. 62

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
I-08	Position Manager Can Hold Its Tokens	Unexpected Behavior	INFORMATIONAL	ACKNOWLEDGED	p. 63
I-09	Ambiguous Error Handling	Error	INFORMATIONAL	ACKNOWLEDGED	p. 64

REMEDIATION REVIEW

H-01	Deposit slippage protection is not sufficient	MEV	HIGH	ACKNOWLEDGED	p. 65
H-02	Whitelisted Address Bypasses Crucial Checks	Logical Error	HIGH	ACKNOWLEDGED	p. 66
H-03	checkTicks Should Exclude Limit Positions	Logical Error	HIGH	ACKNOWLEDGED	p. 67
M-01	Incorrect farming check	DoS	MEDIUM	ACKNOWLEDGED	p. 68
M-02	_detectStuckRewards can block staking	DoS	MEDIUM	RESOLVED	p. 69
L-01	Zero interval TWAP still returns 0	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 70
L-02	Possible Zero Amount Transfer	Logical Error	LOW	ACKNOWLEDGED	p. 72
I-01	token1 can be set as address(0)	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 73
I-02	PRECISION is no longer used	Best Practices	INFORMATIONAL	ACKNOWLEDGED	p. 74

HIGH **H-01**

RESOLVED

No Share Slippage Protection In deposit

CATEGORY	Frontrunning
LOCATION	MultiPositionManager.sol
REVIEW	Main Review

DESCRIPTION

In `MultiPositionManager`, deposits do not enforce a minimum share amount, exposing users to front-running attacks that manipulate pool prices to dilute share allocation.

Attack Scenario:

1. Vault holds equal liquidity in two pools (e.g., USDC/WETH, fee tiers 0.3% and 1%).
2. Bob deposits 1000 USDC and 1 WETH, expecting ~1000 shares based on a balanced vault (assume `total0=10_000 USDC`, `total1=10 WETH`, `totalSupply=10_000`).
3. Alice front-runs Bob by manipulating prices across the two pools:

Swaps USDC → WETH in pool A (increasing WETH price).

Swaps WETH → USDC in pool B (lowering WETH price).

1. This skews the `getTotalAmounts` result:

Total token balances (e.g., `total0 = 20,000 USDC`, `total1 = 20 WETH`) double, but ratio remains balanced.

1. Bob's deposit is now worth a smaller portion of the pool, and he receives only 500 shares instead of 1000.
2. Attacker backruns the deposit and swaps back to bring prices and pool back to original values

As a result, when Bob withdraws he receives less than he deposited losing funds in the process. Depending on the pool type and liquidity available, significant funds could be lost which would go to other depositors of the pool.

RECOMMENDATION

Introduce a `minShares` parameter to `deposit` to ensure depositors receive their expected amount of shares. Also, consider implementing a price deviation check (similar to `HypervisorNFPM`), which would prevent the price manipulation that is needed for this attack vector.

H-01 No Share Slippage Protection In deposit

CONTINUED

PROOF OF CONCEPT <https://github.com/GuardianOrg/MultiPositionManagergamma-posmanager-team1/pull/3/files>

— RESOLUTION

Status: Resolved.

<https://github.com/GammaStrategies/MultiPositionManager/commit/c12e771d79b50de9aecb0fbe3d2e34bfa77d3366>

HIGH **H-02**

RESOLVED

Wrong Usage Of collectRewards Return Values

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/HypervisorNFPM-gammapositionmanagers-team2/blob/4fa8879de218db155b108b395609fc359638c507/contracts/HypervisorNFPM.sol#L597
REVIEW	Main Review

DESCRIPTION

`HypervisorNFPM._collectAndClaim()` incorrectly uses the amounts for the collected pending rewards returned by `farmingCenter.collectRewards()`. It checks if they are a positive number and claims the rewards only if they are. The problem with this approach is that every time an Algebra position enters or exits the farms, any pending rewards are collected. This action can be triggered by anyone via call to `NonFungiblePositionManager.increaseLiquidity()`. The malicious user can add minimum amount of liquidity to the base and limit positions, which will collect any pending rewards. Therefore, if `_collectAndClaim()` is executed without accrual of new rewards, nothing will be claimed. This will disturb the rewards distribution for `MultiFeeDistribution`. For example:

- call `increaseLiquidity` to stop the rewards claiming if huge rewards have accumulated
- `stake` their tokens which will call `Hypervisor.getReward()`, but because both rewards are 0, it will not do anything
- When the rewards are actually sent to the contract, the malicious staker will have received a part of them.

Additional impacts are:

- Rewards being distributed to the wrong `receiver` because there is a `transferReceiver()` function which can change it before the next claim.
- the emitted `RewardsCollected()` event will use wrong values as `reward` and `bonusReward` are pending rewards.

RECOMMENDATION

```
(uint256 reward, uint256 bonusReward) = farmingCenter.collectRewards(key, tokenId);
farmingCenter.collectRewards(key, tokenId);
// Claim and send to MultiFeeDistribution. 0 amount is max amount.
if (reward > 0) {
    farmingCenter.claimReward(key.rewardToken, address(receiver), 0);
    uint256 reward = farmingCenter.claimReward(key.rewardToken, address(receiver), 0);
}
```

H-02 Wrong Usage Of collectRewards Return Values

CONTINUED

```
if (bonusReward > 0) {  
    farmingCenter.claimReward(key.bonusRewardToken, address(receiver), 0);  
    uint256 bonusReward = farmingCenter.claimReward(key.bonusRewardToken, address(receiver), 0);  
}  
emit RewardsCollected(  
    tokenId,  
    reward,  
    bonusReward  
);
```

PROOF OF CONCEPT <https://github.com/GuardianOrg/HypervisorNFPM-gammapositionmanagers-team2/blob/2bffa30051a217086b84afaa5d73b1169eaba0007/test/RebalanceTestFarming.t%20copy.sol#L295-L376>

— RESOLUTION

Status: Resolved.

<https://github.com/GammaStrategies/HypervisorNFPM/commit/af8baa07153bdc61d434562598b43b65f2151afa>

MEDIUM **M-01**

RESOLVED

Unfortunate Withdraw Timing Can Lead To Loss

CATEGORY MEV

LOCATION <https://github.com/GuardianOrg/MultiPositionManagergamma-posmanager-fuzz/blob/48943099b31180c466302da7b2a098e91d284405/src/PoolManagerUtils.sol#L210-L217>

REVIEW Main Review

DESCRIPTION

When a user calls `withdraw`, they provide an `outMin` param which is an array of `amount0/amount1` values that should be returned for each position. If they call `withdraw` and while their transaction is pending, the admin calls `rebalance` and reduces the number of base positions or simply moves the liquidities to different ticks, then the `outMin` array may not correlate to the expected return amounts of the previous positions. Therefore, the slippage checks via the `outMin` parameter can be exploited to steal value from the withdrawal.

Here is an example scenario of when the admin reduces the number of base positions:

- There are 5 base positions. The admin rebalances to 3 base positions.
- The user had previously initiated a `withdraw` transaction with roughly 20% of value expected from each base position.
- Now that there are only 3 base positions, the user should expect to receive ~33% of value from each base position.
- The difference between 33% and 20% can be extracted due to MEV.

RECOMMENDATION

One potential mitigation would be to require the user to input a base positions array into the `withdraw` call that would be checked against the current state of the base positions.

More generally, it may be a good idea to enforce a deadline parameter so that the users' `withdraw` transaction cannot stay pending, potentially while the base positions array is updated due to rebalances.

RESOLUTION

Status: Resolved.

<https://github.com/GammaStrategies/MultiPositionManager/commit/73f92a8fb44839bda5ae53ceb3fa1cacaf2956d3>

MEDIUM **M-02**

RESOLVED

Fee Checked Incorrectly

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/MultiPositionManagergamma-posmanager-fuzz/blob/48943099b31180c466302da7b2a098e91d284405/src/MultiPositionManager.sol#L326
REVIEW	Main Review

DESCRIPTION

When setting the fee that is sent to the treasury, the new fee value is checked incorrectly. Instead, the previous fee value is checked to ensure that it is not less than one.

```
if (fee < 1) revert InvalidFee();
```

RECOMMENDATION

Instead, check that the `newFee` parameter is not less than one and revert if it is.

RESOLUTION

Status: Resolved.

<https://github.com/GammaStrategies/MultiPositionManager/commit/73f92a8fb44839bda5ae53ceb3fa1cacaf2956d3>

MEDIUM **M-03**

RESOLVED

Tick Bounds Can Be Miscalculated

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/MultiPositionManagergamma-posmanager-team1/blob/48943099b31180c466302da7b2a098e91d284405/src/ClearingV3.sol#L138
REVIEW	Main Review

DESCRIPTION

The `deposit` function in the `UniProxyV2` contract calls `clearance.clearDeposit` to ensure all positions have their current tick within the allowed range. If any are out of range, the call reverts.

However, when there are no limit positions—such as when `limitWidth` is set to zero during a rebalance—the `_checkTicks` function receives zero values for those positions from `getPositions`. These zero-value entries are still included in the loop that determines the lowest and highest ticks.

As a result, they can be incorrectly selected as the min or max tick, distorting the tick bounds. This may cause a real position with an out-of-range tick to appear valid, allowing the deposit to proceed when it should have reverted.

This undermines the tick range enforcement and introduces the risk of deposits occurring outside of the configured bounds.

RECOMMENDATION

In the `getPositions` and `currentTicks` functions, return early if limit positions are not set, similar to how this case is handled elsewhere in the contract.

RESOLUTION

Status: Resolved.

<https://github.com/GammaStrategies/MultiPositionManager/commit/73f92a8fb44839bda5ae53ceb3fa1cacaf2956d3>

MEDIUM **M-04**

RESOLVED

Overflow In Price Calculation Not Fixed

CATEGORY Math

LOCATION <https://github.com/GuardianOrg/HypervisorNFPM-gammapositionmanagers-team1/blob/main/contracts/ClearingV3NFPM.sol#L191>

REVIEW Main Review

DESCRIPTION

In the previous audit, Issue 31 raised the possibility of overflow for pairs with very large price.

The original price calculation was:

```
uint256 price = FullMath.mulDiv(uint256(sqrtPrice) * uint256(sqrtPrice),  
PRECISION, 2**(96 * 2));
```

which was fixed to:

```
FullMath.mulDiv(uint256(sqrtPrice) * 1e18, uint256(sqrtPrice) * 1e18,  
2**(96 * 2));
```

This was fixed in the `_deposit` function of `HypervisorNFPM.sol`. However, the original calculation still exists in the `checkPriceChange` function of `ClearingV3NFPM.sol`. By using the original implementation, overflow can occur when performing:

```
uint256(sqrtPriceBefore) * uint256(sqrtPriceBefore).
```

RECOMMENDATION

Update the price calculation `ClearingV3NFPM.sol` to the corrected version which prevents overflow.

RESOLUTION

Status: Resolved.

<https://github.com/GammaStrategies/HypervisorNFPM/commit/af8baa07153bdc61d434562598b43b65f2151afa#diff-b04add23b4a22fcfa7526395cbc5a7895867eaf2d2a55fa57fcee1d3e2a1b0fc>

MEDIUM **M-05**

ACKNOWLEDGED

Algebra Oracle Vulnerable To TWAP Lookback DoS

CATEGORY DoS

LOCATION <https://github.com/GuardianOrg/HypervisorNFPM-gammapositionmanagers-team1/blob/main/contracts/ClearingV3NFPM.sol#L210>

REVIEW Main Review

DESCRIPTION

In HypervisorNFPM, a TWAP-based price check is performed before deposits, using Algebra's Volatility Oracle. The oracle computes TWAP via stored timepoints, which are indexed by a `uint16 timepointIndex` with a maximum of 65,535 entries (`uint16.max`).

Timepoints are recorded during swaps, but only once per block. If a swap occurs in a block and a timepoint is recorded, any subsequent swaps in the same block won't create additional entries. Over time, once the `timepointIndex` overflows and older timepoints are overwritten.

The issue lies with how if the caller specifies a `twapInterval` that is older than the oldest stored timepoint, the `_getTimepointsAt` function will revert (<https://github.com/cryptotalgebra/Algebra/blob/51e5f9697a394237de17ae5892ae1c7575d7f8fc/src/plugin/contracts/libraries/VolatilityOracle.sol#L464>)

Attack vector:

If an attacker triggers a swap in every block, the oracle fills up and starts overwriting past timepoints. On chains like BSC, where the average block time is ~0.75s, the full 65,535 timepoint buffer can be filled in:

$$0.75s * 65,535 \approx 49,000 \text{ seconds} \approx 13.6 \text{ hours}$$

The HypervisorNFPM vault uses a `twapInterval` of 86,400 seconds (24 hours) for stable pools. This interval will eventually become unreachable in the oracle, causing TWAP reads to revert and permanently blocking deposits unless the interval is reduced.

RECOMMENDATION

Constrain `twapInterval` to a safe maximum, calculated as:

`twapInterval = avg block time * 65,535`. Alternatively, consider using a non-twap oracle such as Chainlink to obtain price.

M-05 Algebra Oracle Vulnerable To TWAP Lookback DoS

CONTINUED

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-06**

RESOLVED

Rebalance Can Revert During Emergency Mode

CATEGORY DoS

LOCATION <https://github.com/GuardianOrg/HypervisorNFPM-gammapositionmanagers-team1/blob/4fa8879de218db155b108b395609fc359638c507/contracts/HypervisorNFPM.sol#L521>

REVIEW Main Review

DESCRIPTION

The `rebalance` function burns and re-mints positions, then calls `_approveAndEnterFarming` to approve and enter the new farming position. However, `_approveAndEnterFarming` does not check whether `isEmergencyWithdrawActivated` is enabled in the `AlgebraEternalFarming` contract.

If emergency mode is active, `enterFarming` will revert inside `farmingCenter`, blocking the entire rebalance process. Without rebalancing, the protocol cannot maintain its intended strategy. Furthermore, if the current tick moves out of range, certain functions will be blocked due to implemented checks—such as `deposit`, which requires the current tick to be in range.

RECOMMENDATION

Add a check for `isEmergencyWithdrawActivated()` in `_approveAndEnterFarming`, and return early if true, similar to how the `isIncentiveDeactivated` check is implemented.

RESOLUTION

Status: Resolved.

<https://github.com/GammaStrategies/HypervisorNFPM/commit/af8baa07153bdc61d434562598b43b65f2151afa#diff-a551d62ef5d903aa709b2186efce87c30b93e912fd1cc5982725026e2daf1dd3>

MEDIUM **M-07**

RESOLVED

Clearing Reverts On Inactive Limit Positions

CATEGORY DoS

LOCATION <https://github.com/GuardianOrg/MultiPositionManagergamma-posmanager-team2/blob/48943099b31180c466302da7b2a098e91d284405/src/ClearingV3.sol#L160-L169>

REVIEW Main Review

DESCRIPTION

`Clearing._checkTicks()` ensures that each tick in `currentTicks` is between `lowestTick` and `highestTick`, which are derived from all base and limit positions. However, even inactive limit positions (e.g., when `limitWidth == 0`) are included in `currentTicks`. These inactive positions have uninitialized pool keys, defaulting their ticks to 0.

As a result, if `lowestTick > 0` or `highestTick <= 0`, `_checkTicks()` will incorrectly revert, causing a denial of service for deposits.

RECOMMENDATION

You can change `MultiPositionManager.currentTicks()` to return a magic value for inactive positions and then skip the iteration in the for loop of `_checkTicks` if the `currentTick` is equal to the magic value.

RESOLUTION

Status: Resolved.

<https://github.com/GammaStrategies/MultiPositionManager/commit/73f92a8fb44839bda5ae53ceb3fa1cacaf2956d3>

MEDIUM **M-08**

ACKNOWLEDGED

decreaseLiquidity Doesn't Collect Funds

CATEGORY Unexpected Behavior

LOCATION <https://github.com/GuardianOrg/HypervisorNFPM-gammapositionmanagers-team2/blob/4fa8879de218db155b108b395609fc359638c507/contracts/HypervisorNFPM.sol#L722-L735>

REVIEW Main Review

DESCRIPTION

`HypervisorNFPM` has two functions that allow the owner to manually control positions on behalf of the contract - `mintLiquidity` and `decreaseLiquidity`. The first one can be used to create new positions by adding liquidity and using them to farm incentives. The second one is used to pull liquidity from positions to the contract.

The problem is that `decreaseLiquidity` doesn't call `_zeroBurn` and `farmingCenter.collectRewards()`. This means that:

- if the position being decreased is `base` or `limit` the generated fees will be collected together with the removed liquidity and all of them will be attributed to the vault participants, i.e no cut for the protocol team.
- if the position being decreased is not `base` or `limit`, all the incentives generated in the farm will be stuck there as there is no way for the contract to call `collectRewards` with the id of that position. An exception is if the owner calls `setNftIds` and sets the position as either `base` or `limit` to claim the rewards, but this can disrupt the normal flow of the contract.

RECOMMENDATION

Call `_zeroBurn()` at the beginning of `decreaseLiquidity` and also collect the received incentives if any.

RESOLUTION

Status: Acknowledged.

LOW **L-01**

ACKNOWLEDGED

Excess Native Refunds May Revert For Contracts

CATEGORY Best Practices

LOCATION <https://github.com/GuardianOrg/MultiPositionManagergamma-posmanager-fuzz/blob/48943099b31180c466302da7b2a098e91d284405/src/MultiPositionManager.sol#L606-L615>

REVIEW Main Review

— DESCRIPTION

In the `_transferIn` function, if someone sends too much native currency (like ETH), the contract tries to refund the extra using `.transfer`. While that works fine for normal wallets (EOAs), it becomes a problem if the sender is a contract. The `.transfer` method only forwards 2300 gas, which isn't enough for contracts that need more gas in their `receive()` or `fallback()` functions. So if the sender is a contract that expects to do anything when receiving ETH, this refund logic will revert.

— RECOMMENDATION

Instead of using `.transfer`, use `.call{value: ...}()` to forward the ETH.

```
(bool success, ) = payable(msg.sender).call{value: msg.value - amount}("");  
require(success, "Refund failed");
```

— RESOLUTION

Status: Acknowledged.

LOW **L-02**

ACKNOWLEDGED

FeeRecipient Update Can Be Frontrun

CATEGORY Documentation

LOCATION <https://github.com/GuardianOrg/HypervisorNFPM-gamma/positionmanagers-fuzz/blob/988b54bb641733c5e8307cfeb7dd127021a5449b/contracts/HypervisorNFPM.sol#L342>

REVIEW Main Review

DESCRIPTION

In `HypervisorNFPM.sol`, the admin sets the `feeRecipient` in each call to `rebalance()`. The comment indicates `_feeRecipient Address of recipient of % of fees since last rebalance`, however this will not always be true.

If a user performs any action such as a deposit or withdraw, it will call `_zeroBurn()` which will collect the fees and transfer them to the current `feeRecipient`.

RECOMMENDATION

Note this or update the comment to indicate that the fees may be received to the currently set address in the case of user deposits or withdrawals.

RESOLUTION

Status: Acknowledged.

LOW **L-03**

RESOLVED

Unnecessary Approvals

CATEGORY	Unexpected Behavior
LOCATION	https://github.com/GuardianOrg/MultiPositionManagergamma-posmanager-fuzz/blob/48943099b31180c466302da7b2a098e91d284405/src/ClearingV3.sol#L51-L54
REVIEW	Main Review

DESCRIPTION

The ClearingV3.sol contract performs token approvals but does not perform any transfers.

RECOMMENDATION

Remove the token approval logic to avoid confusion of the contract's purpose.

RESOLUTION

Status: Resolved.

<https://github.com/GammaStrategies/MultiPositionManager/commit/73f92a8fb44839bda5ae53ceb3fa1cacaf2956d3>

LOW **L-04**

RESOLVED

Unused ReentrancyGuard In ClearingV3.sol

CATEGORY Informational**LOCATION** <https://github.com/GuardianOrg/MultiPositionManagergamma-posmanager-fuzz/blob/48943099b31180c466302da7b2a098e91d284405/src/ClearingV3.sol#L13>**REVIEW** Main Review

DESCRIPTION

The ReentrancyGuard.sol contract is inherited but not used in the ClearingV3.sol.

RECOMMENDATION

Remove the inherited contract to avoid confusion and excess code size.

RESOLUTION

Status: Resolved.

<https://github.com/GammaStrategies/MultiPositionManager/commit/73f92a8fb44839bda5ae53ceb3fa1cacaf2956d3>

LOW **L-05**

RESOLVED

Incorrect Error Usage

CATEGORY Informational

LOCATION <https://github.com/GuardianOrg/HypervisorNFPM-gammapositionmanagers-fuzz/blob/ca0fa1cbd65cf632f747f1f6db016c7c0ce95445/contracts/MultiFeeDistribution.sol#L103C56-L103C67>

REVIEW Main Review

DESCRIPTION

`InvalidBurn()` used in check for reward tokens,

RECOMMENDATION

TBD

RESOLUTION

Status: Resolved.

<https://github.com/GammaStrategies/HypervisorNFPM/commit/af8baa07153bdc61d434562598b43b65f2151afa#diff-cefd2887e842ad0378b8ef8a02023780d42564c9abf55dadd929a5e83cabf620>

LOW **L-06**

ACKNOWLEDGED

Incorrect Price For Negative Tick

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/HypervisorNFPM-gammapositionmanagers-team1/blob/4fa8879de218db155b108b395609fc359638c507/contracts/ClearingV3NFPM.sol#L200
REVIEW	Main Review

DESCRIPTION

The `getSqrtTwapX96` function in the `ClearingV3NFPM` contract computes a square-root TWAP over a specified interval. However, when the tick difference is negative and not evenly divisible by the `_twapInterval`, the function fails to round the tick down as required. Instead, it truncates toward zero, resulting in an incorrect (too high) tick value and thus an inaccurate price

RECOMMENDATION

Implement rounding toward negative infinity when the tick difference is negative and has a remainder. Specifically, if `tickCumulatives[1] - tickCumulatives[0]` is negative and `(delta % _twapInterval) != 0`, decrement the tick by one.

RESOLUTION

Status: Acknowledged.

LOW **L-07**

ACKNOWLEDGED

mintLiquidity Mishandles Fees

CATEGORY	Unexpected Behavior
LOCATION	https://github.com/GuardianOrg/HypervisorNFPM-gammapositionmanagers-team1/blob/main/contracts/HypervisorNFPM.sol#L573
REVIEW	Main Review

DESCRIPTION

The `mintLiquidity` function creates a new liquidity position with a fresh `tokenId`. While this new position is entered into a farming position, it is not accessible by `MultiFeeDistribution` for reward claims.

Currently, `getReward` only claims rewards for the base and limit `tokenIds` set in the vault. As a result, rewards for the newly minted position remain unclaimable until the vault owner explicitly calls `setNftIds` to update one of the tracked token IDs.

On a similar note, when `setNftIds` is called, the rewards for the current base and limit positions are not collected first. As result, these rewards are lost once the new `tokenIds` are written.

RECOMMENDATION

1. Consider implementing a `getReward` function that allows for the passing in of a `tokenId` to allow rewards to be claimed for additional positions beyond the base and limit positions.
2. Consider calling `getReward` on the current base and limit positions before `setNftIds` updates.

RESOLUTION

Status: Acknowledged.

LOW **L-08**

ACKNOWLEDGED

getTotalAmountsPlusFees May Be Stale

CATEGORY Warning

LOCATION <https://github.com/GuardianOrg/HypervisorNFPM-gamma-positionmanagers-team1/blob/4fa8879de218db155b108b395609fc359638c507/contracts/HypervisorNFPM.sol#L457>

REVIEW Main Review

— DESCRIPTION

The `getTotalAmountsPlusFees` function returns the total of `token0` and `token1` including fees, using `calculatePositionFee` to compute the latest fees and add them to the currently owed amounts.

However, this doesn't account for how Algebra pools handle rebase tokens/donations. At the start of major pool interactions, if the actual token balances exceed expected reserves, the surplus is treated as a donation and distributed to active liquidity providers as additional fees—provided there is non-zero liquidity. Since `getTotalAmountsPlusFees` doesn't account for this, these additional fees may not be reflected in the returned values.

While this isn't an issue for core HypervisorNFPM functions (which call `zeroBurn` first), it could lead to stale or slightly inaccurate fee values when calling `getTotalAmountsPlusFees` directly.

— RECOMMENDATION

Be aware that `getTotalAmountsPlusFees` may be stale or slightly inaccurate in the following scenario.

— RESOLUTION

Status: Acknowledged.

LOW **L-09**

RESOLVED

Possible 0 Token Transfer

CATEGORY	Rewards
LOCATION	https://github.com/GuardianOrg/HypervisorNFPM-gammapositionmanagers-team2/blob/4fa8879de218db155b108b395609fc359638c507/contracts/MultiFeeDistribution.sol#L462
REVIEW	Main Review

DESCRIPTION

`MultiFeeDistribution._updateReward()` transfers `newRewards / fee` as a fee to the owner unconditionally. If one of the reward tokens is a token that reverts on 0 transfers, users can grief the actions of the contract by sending as little as 1 wei of that token (if this token wasn't accrued currently).

RECOMMENDATION

Check if the amount after the division is positive

```
- if (newRewards > 0) {  
+ if (newRewards / fee > 0) {  
  IERC20(rewardToken).safeTransfer(owner(), newRewards/fee);  
  currentBalance = IERC20(rewardToken).balanceOf(address(this));  
  newRewards = newRewards - (newRewards/fee); // Actual new rewards after fee  
}
```

RESOLUTION

Status: Resolved.

<https://github.com/GammaStrategies/HypervisorNFPM/commit/af8baa07153bdc61d434562598b43b65f2151afa#diff-cefd2887e842ad0378b8ef8a02023780d42564c9abf55dadd929a5e83cabf620>

LOW **L-10**

ACKNOWLEDGED

setNftIds Doesn't Collect Funds

CATEGORY	Unexpected Behavior
LOCATION	https://github.com/GuardianOrg/HypervisorNFPM-gammapositionmanagers-team2/blob/4fa8879de218db155b108b395609fc359638c507/contracts/HypervisorNFPM.sol#L709-L713
REVIEW	Main Review

DESCRIPTION

`HypervisorNFPM.setNftIds()` is a helpful utility function which can be used by the owner to change any of the two position ids. Similar to the issue described in the other report about `decreaseLiquidity()`, `setNftIds` doesn't call `_zeroBurn()` and `_collectAndClaim()`. The following problem exists If the function is used to change the ids for a prolonged period of time - accrued fees and incentive rewards will be either lost if the old ids are not restored or they will be wrongly distributed because they will not be counted towards the total funds for any new deposits in the window where `setNftId` took place.

RECOMMENDATION

Call `_zeroBurn()` and `_collectAndClaim()` before changing the ids.

RESOLUTION

Status: Acknowledged.

LOW **L-11**

ACKNOWLEDGED

Unnecessary Check In addReward()

CATEGORY Best Practices

LOCATION <https://github.com/GuardianOrg/HypervisorNFPM-gammapositionmanagers-team2/blob/4fa8879de218db155b108b395609fc359638c507/contracts/MultiFeeDistribution.sol#L155-L161>

REVIEW Main Review

DESCRIPTION

`MultiFeeDistribution.addReward()` checks if the token to be added is an active reward twice, which is unnecessary and only causes higher gas expenditure.

```
(bool isRewardTokenExist, ) = _isRewardTokenExist(_rewardToken);
if (isRewardTokenExist) revert ActiveReward();
if (!managers[msg.sender]) revert InsufficientPermission();
for (uint i; i < rewardTokens.length; i++) {
    if (rewardTokens[i] == _rewardToken) revert ActiveReward();
}
```

RECOMMENDATION

Remove one of the checks, for example the first one since it's more expensive.

```
- (bool isRewardTokenExist, ) = _isRewardTokenExist(_rewardToken);
- if (isRewardTokenExist) revert ActiveReward();
if (!managers[msg.sender]) revert InsufficientPermission();
for (uint i; i < rewardTokens.length; i++) {
    if (rewardTokens[i] == _rewardToken) revert ActiveReward();
}
rewardTokens.push(_rewardToken);
```

RESOLUTION

Status: Acknowledged.

LOW **L-12**

ACKNOWLEDGED

simulateOneSecondGrowth Is Not Used

CATEGORY Best Practices
LOCATION RewardCalculations.sol
REVIEW Main Review

DESCRIPTION

The `RewardCalculations.simulateOneSecondGrowth()` function is never called in the codebase.

RECOMMENDATION

Consider removing it.

RESOLUTION

Status: Acknowledged.

LOW **L-13**

ACKNOWLEDGED

Slippage Not Enforced For Limit Positions

CATEGORY	Frontrunning
LOCATION	https://github.com/GuardianOrg/MultiPositionManageregamma-posmanager-team1/blob/main/src/PoolManagerUtils.sol#L251 , https://github.com/GuardianOrg/MultiPositionManageregamma-posmanager-team1/blob/main/src/PoolManagerUtils.sol#L124
REVIEW	Main Review

DESCRIPTION

The `inMin` and `outMin` parameters are user-supplied slippage controls used during `withdraw`, `rebalance` and `compound` to ensure the user or protocol uses or receives a minimum acceptable amount of tokens when adding or removing liquidity.

However, this check is not applied when minting or burning liquidity from limit positions. Instead, `_mintLiquidityForAmounts` and `burnLiquidityForShare` are called with hardcoded zero slippage tolerance:

```
(uint256 amountOut0, uint256 amountOut1) = burnLiquidityForShare(
    poolManager,
    limitPositions[i],
    shares,
    totalSupply,
    [uint256(0), uint256(0)] // @audit zero slippage tolerance
);
```

This bypasses the intended slippage protection and opens up the vault to price manipulation or frontrunning attacks, where:

- A user may receive significantly less than expected during a `withdraw`.
- The vault may unintentionally deploy liquidity in a distorted ratio during `rebalance` or `compound`.

RECOMMENDATION

1. Apply `inMin` and `outMin` by passing the relevant slippage values into `_mintLiquidityForAmounts` and `burnLiquidityForShare` for limit positions instead of `[0, 0]`.
2. In `UniProxyV2`, `getOutMinForShares` should also calculate `outMin` for limit positions.

L-13 Slippage Not Enforced For Limit Positions

CONTINUED

— **RESOLUTION**

Status: Acknowledged.

LOW **L-14**

ACKNOWLEDGED

Unnecessary Assignment

CATEGORY Best Practices

LOCATION <https://github.com/GuardianOrg/HypervisorNFPM-gammapositionmanagers-team2/blob/4fa8879de218db155b108b395609fc359638c507/contracts/HypervisorNFPM.sol#L84>

REVIEW Main Review

DESCRIPTION

The constructor of `HyperVisorNFPM` sets the value of `incentiveMaker` twice - once outside the if statement and once inside of it - to the same value, which is redundant.

RECOMMENDATION

Delete the second assignment.

RESOLUTION

Status: Acknowledged.

LOW **L-15**

RESOLVED

Staking Token Can Be Set As Reward Token

CATEGORY Best Practices

LOCATION <https://github.com/GuardianOrg/HypervisorNFPM-gammapositionmanagers-team2/blob/4fa8879de218db155b108b395609fc359638c507/contracts/MultiFeeDistribution.sol#L141-L145>

REVIEW Main Review

DESCRIPTION

`MultiFeeDistribution.setStakingToken()` doesn't check if `_stakingToken` is not an active reward token. This allows bypassing the check in `addReward()` and adding a reward token as a staking token.

RECOMMENDATION

Consider reverting the `setStakingToken()` transaction if `_stakingToken` is an active reward token.

RESOLUTION

Status: Resolved.

<https://github.com/GammaStrategies/HypervisorNFPM/commit/af8baa07153bdc61d434562598b43b65f2151afa#diff-cefd2887e842ad0378b8ef8a02023780d42564c9abf55dadd929a5e83cabf620>

LOW **L-16**

ACKNOWLEDGED

Clearing Doesn't Consider Liquidity

CATEGORY Informational
LOCATION Clearing.sol
REVIEW Main Review

— DESCRIPTION

When ticks are checked in `Clearing`, all positions are included, even inactive ones (i.e. `liquidity = 0`). It may be possible that some positions have 0 liquidity after withdrawals or if there wasn't enough liquidity when limit positions were created. These positions will still impact the clearing validation.

— RECOMMENDATION

Make sure that's the correct behavior.

— RESOLUTION

Status: Acknowledged.

LOW **L-17**

ACKNOWLEDGED

CEI Pattern Not Followed

CATEGORY	Best Practices
LOCATION	MultiPositionManager.sol
REVIEW	Main Review

— DESCRIPTION

`MultiPositionManager.deposit()` and `MultiPositionManager.withdraw()` functions doesn't follow the `CEI` pattern - they transfer tokens before writing to the contract state. This gives the execution flow the recipient for pools with native token or pools with tokens with hooks. This opens up possibilities for read-only reentrancy and unexpected behavior. For example, a user may use the fund transfer in `withdraw()` to transfer more tokens to the `MultiPositionManager` and cause wrong `Withdraw()` event data emission.

— RECOMMENDATION

Consider following the `CEI` pattern.

— RESOLUTION

Status: Acknowledged.

LOW **L-18**

ACKNOWLEDGED

Withdrawals Can Be Weaponized

CATEGORY Unexpected Behavior
LOCATION MultiPositionManager.sol
REVIEW Main Review

— DESCRIPTION

When users withdraw, funds are taken out of the base and limit positions. Because there isn't any delay between depositing and minting, users can atomically withdraw their deposits and pull liquidity out of the positions and grieving the manager from collecting fees.

— RECOMMENDATION

Consider implementing a delay between deposit and withdraw

— RESOLUTION

Status: Acknowledged.

LOW **L-19**

RESOLVED

Potential 0 Transfer On Withdraw

CATEGORY Validation

LOCATION <https://github.com/GuardianOrg/MultiPositionManagergamma-posmanager-team2/blob/48943099b31180c466302da7b2a098e91d284405/src/MultiPositionManager.sol#L201-L202>

REVIEW Main Review

DESCRIPTION

`MultiPositionManager.withdraw()` transfers `amount0` and `amount1` of the currencies unconditionally, even if they are zeroes. If used with some tokens that revert on 0 transfers, withdrawals may be blocked.

RECOMMENDATION

Skip the transfers if the amounts are 0s.

RESOLUTION

Status: Resolved.

<https://github.com/GammaStrategies/MultiPositionManager/commit/b7667f70fbb3a2dd71d0b36ab7cac0368608ff6f>

LOW **L-20**

RESOLVED

ETH Surplus May Not Be Refunded

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/MultiPositionManagergamma-posmanager-team2/blob/48943099b31180c466302da7b2a098e91d284405/src/MultiPositionManager.sol#L607
REVIEW	Main Review

DESCRIPTION

In the `MultiPositionManager.deposit()` function, when interacting with pools where `token0 == address(0)` (i.e., native token), the `_transferIn()` function is responsible for handling incoming native tokens and refunding any excess via the `msg.value > amount` check.

However, a logic flaw exists: if `amount == 0`, `_transferIn()` returns early without executing the refund logic. This results in a scenario where any native tokens sent along with the call are silently retained, causing a loss of funds for the user.

This can happen when:

- The pool's current tick is above the upper bound of both the base and limit positions, making the `total0 == 0`.
- The deposit is contributing only token1, so `amount == 0` is passed to `_transferIn()`.
- There are no idle token0 funds or pending fees in the contract, so no internal balance offsets this.
- The user unintentionally sends a nonzero `msg.value`, expecting a partial refund.

Currently, this situation is prevented by the `Clearing` contract, which blocks such deposits. However, future changes to the clearing logic or the whitelist may reopen this vulnerability.

RECOMMENDATION

Don't skip the refund for native tokens.

```
function _transferIn(address from, Currency currency, uint256 amount) internal {  
    - if (amount == 0) return;  
    if (currency.isAddressZero()) {  
        if (msg.value < amount) revert InvalidDepositAmount(amount);  
        if (msg.value > amount)  
            payable(msg.sender).transfer(msg.value - amount);  
    } else if (amount != 0) {  
        IERC20(Currency.unwrap(currency)).safeTransferFrom(from, address(this), amount);  
    }
```

L-20 ETH Surplus May Not Be Refunded

CONTINUED

`}``}`

PROOF OF CONCEPT <https://github.com/GuardianOrg/MultiPositionManagergamma-posmanager-team2/pull/2>

— RESOLUTION

Status: Resolved.

<https://github.com/GammaStrategies/MultiPositionManager/commit/b7667f70fbb3a2dd71d0b36ab7cac0368608ff6f>

LOW **L-21**

ACKNOWLEDGED

inMin Is Not Enforced When liquidity = 0

CATEGORY Validation

LOCATION <https://github.com/GuardianOrg/MultiPositionManagergamma-posmanager-team2/blob/48943099b31180c466302da7b2a098e91d284405/src/PoolManagerUtils.sol#L188-L193>

REVIEW Main Review

— DESCRIPTION

`PoolManagerUtils._mintLiquidityForAmounts()` enforces the `inMin` constraint only when `liquidity > 0`. However, `mintLiquidity()` may call `_mintLiquidityForAmounts()` with a positive `liquidity` value but with actual token amounts equal to zero—for instance, if the contract's token balances were already depleted before this call.

In such a case, the function skips minting (since `liquidity == 0`), and no check is performed against `inMin`, even if the expected minimum input amount was positive. This leads to a silent mismatch between intended and actual behavior.

— RECOMMENDATION

Consider validating the added amounts against `inMin` even if there was no liquidity added.

— RESOLUTION

Status: Acknowledged.

LOW **L-22**

ACKNOWLEDGED

UniProxy.transferETH() Is Permissionless

CATEGORY Informational

LOCATION <https://github.com/GuardianOrg/MultiPositionManagergamma-posmanager-team2/blob/48943099b31180c466302da7b2a098e91d284405/src/UniProxyV2.sol#L90-L93>

REVIEW Main Review

DESCRIPTION

The `UniProxy.transferETH()` function used for making ETH transfers is external and permissionless which means any funds in the contract can be taken out by users.

RECOMMENDATION

Never hold funds in the contract.

RESOLUTION

Status: Acknowledged.

LOW **L-23**

RESOLVED

Whitelisted Depositors Are Not Exempted

CATEGORY	Validation
LOCATION	https://github.com/GuardianOrg/MultiPositionManagergamma-posmanager-team2/blob/48943099b31180c466302da7b2a098e91d284405/src/UniProxyV2.sol#L55
REVIEW	Main Review

DESCRIPTION

The `ClearingV3` contract contains a list with depositors (`freeDepositList`) that should not be subject of the clearing logic. However, this list is never utilized and in result the whitelisted entities are no different than every other user.

RECOMMENDATION

Consider executing `clearDeposit` in `UniProxyV2` only if `msg.sender` is not whitelisted.

```
+ if (!clearance.getListed(pos, msg.sender)) {  
clearance.clearDeposit(to, pos);  
+ }
```

RESOLUTION

Status: Resolved.

<https://github.com/GammaStrategies/MultiPositionManager/commit/b7667f70fbb3a2dd71d0b36ab7cac0368608ff6f>

LOW **L-24**

ACKNOWLEDGED

UniProxyETH Is Vulnerable To Tokens With Hooks

CATEGORY	Unexpected Behavior
LOCATION	UniProxyETH.sol
REVIEW	Main Review

DESCRIPTION

`UniProxyETH.depositETH()` and `UniProxyETH.depositETHAndStake()` perform these three actions in the following order:

- `clearDeposit()`
- transfer token from user
- execute the `Hypervisor` deposit

If the token to be transferred has hooks, the depositor will be able to bypass the clearing mechanism, including manipulating the price outside of the bounds of the `TWAP` and effectively draining the contract.

RECOMMENDATION

Move the token transfer from the second step in the description before the call to `clearDeposit()` and use `deposit0` and `deposit1` as transferred amounts. This is okay since there is a refund mechanism at the end of the function.

RESOLUTION

Status: Acknowledged.

LOW **L-25**

RESOLVED

Zero-Interval TWAP Always Returns Zero

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/HypervisorNFPM-gammapositionmanagers-team1/blob/4fa8879de218db155b108b395609fc359638c507/contracts/ClearingV3NFPM.sol#L200
REVIEW	Main Review

DESCRIPTION

The `getSqrtTwapX96` function in the `ClearingV3NFPM` contract is intended to return the square-root TWAP price over a specified interval. When the interval is zero, it should return the current price. However, while the current price is fetched, it is not assigned to the function's return variable. As a result, calls with a zero interval always return zero.

Consequently, when `checkPriceChange` uses this function, a zero interval leads to a zero price. This causes the price-change threshold check to be exceeded and the function to revert, effectively blocking deposits.

RECOMMENDATION

Fix `getSqrtTwapX96` so that when the interval is zero, the fetched current price is correctly assigned to the return variable

RESOLUTION

Status: Resolved.

<https://github.com/GammaStrategies/HypervisorNFPM/commit/af8baa07153bdc61d434562598b43b65f2151afa#diff-b04add23b4a22fcfa7526395cbc5a7895867eaf2d2a55fa57fcee1d3e2a1b0fc>

LOW **L-26**

ACKNOWLEDGED

Misaligned Limit Positions Due To slot0.tick

CATEGORY	Unexpected Behavior
LOCATION	https://github.com/GuardianOrg/MultiPositionManagergamma-posmanager-team1/blob/main/src/MultiPositionManager.sol#L707
REVIEW	Main Review

DESCRIPTION

In `MultiPositionManager`, limit positions are centered around `slot0.tick`, assuming it reflects the current price. However, this is inaccurate when a swap ends exactly at a tick boundary. As per Uniswap's swap logic, when `result.sqrtPriceX96 == step.sqrtPriceNextX96` at the end of a swap step, and the direction is `zeroForOne`, the protocol sets the current tick to `tickNext - 1` (source) [<https://github.com/Uniswap/v4-core/blob/main/src/libraries/Pool.sol#L431>].

Example scenario:

- Initial setup: Stable pool tick range `[-20, 20]`, current tick: `0`, initial price: `X`
- A swap token1 -> token0 moves tick to `1`
- An equal swap back from token0 -> token1 brings price back to exactly `X`
- However, current tick is now `-1`, not `0` due to Uniswap's handling of `zeroForOne` swaps

As a result, limit positions of 1 tick width are deployed at range `[-2, 0]`, leading to inefficient limit position placement and therefore missed fee opportunities.

RECOMMENDATION

Consider allowing admin to specify exactly which tick and ranges to deploy limit liquidity in (similar to how HypervisorNFPM is implemented).

Alternatively, use `slot0.sqrtPriceX96` to derive the actual current tick using `TickMath.getSqrtPriceAtTick()`. This ensures limit positions are accurately centered at the true price.

PROOF OF CONCEPT <https://github.com/GuardianOrg/MultiPositionManagergamma-posmanager-team1/pull/4/files>

RESOLUTION

Status: Acknowledged.

LOW **L-27**

ACKNOWLEDGED

Inconsistent Price Threshold Check

CATEGORY Math

LOCATION <https://github.com/GuardianOrg/HypervisorNFPM-gammapositionmanagers-team2/blob/4fa8879de218db155b108b395609fc359638c507/contracts/ClearingV3NFPM.sol#L192-L193>

REVIEW Main Review

DESCRIPTION

`ClearingV3NFPM.checkPriceChange()` compares the Algebra pool's spot price to a given TWAP price and reverts if the price deviates more than a set threshold. While the computation works when `spotPrice > twapPrice`, it fails in the opposite case - it calculates the price change as if the price went from `spotPrice` to `twapPrice` instead of `twapPrice` to `spotPrice`. The code doing the math is the following:

```
if (price * 10_000 / priceBefore > _priceThreshold || priceBefore * 10_000 / price >
_priceThreshold)
    revert("Price change overflow");
```

Let's say:

- `priceBefore = $200` (TWAP price)
- `price = $100` (Spot price)

This is a 50% deviation from the `TWAP`. However, value checked against `_priceThreshold` will be the maximum between

1. `price * 10_000 / priceBefore = 100 * 10000 / 200 = 5000`
2. `priceBefore * 10_000 / price = 200 * 10000 / 100 = 20000` (100%)

In result, the price change from `$200` to `$100` will be considered as a `100%` change instead of `50%`.

RECOMMENDATION

Compute an absolute delta value between the two prices and calculate the deviation as `delta * 10_000 / priceBefore + 10_000`.

RESOLUTION

Status: Acknowledged.

LOW **L-28**

RESOLVED

Lack Of Checkpointing When Setting New Fee

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/HypervisorNFPM-gammapositionmanagers-fuzz/blob/9919c2000cf27691590fd9046fcfcdb6bafcc784/contracts/HypervisorNFPM.sol#L656-L660
REVIEW	Main Review

DESCRIPTION

In the Hypervisor contract, the admin is able to set the percentage of fees that will be collected as protocol fees. However, the new value is set without firstly calling `_zeroBurn()` which transfers the relevant fee values to the `feeRecipient`.

Therefore, The admin can set this value to its lowest value, e.g. 1, which will transfer them all fees that are collected.

Contrast this to the `MultiPositionManager` contract which does successfully call `_zeroBurn()` prior to setting the new fee.

Similarly, the fees are not checkpointed in `MultiFeeDistribution.sol` either when resetting the fee.

RECOMMENDATION

Call `_zeroBurn()` prior to updating the fee value in `HypervisorNFPM.sol`. Call `_updateRewards()` in `MultiFeeDistribution.sol`.

RESOLUTION

Status: Resolved.

<https://github.com/GammaStrategies/HypervisorNFPM/commit/af8baa07153bdc61d434562598b43b65f2151afa#diff-cefd2887e842ad0378b8ef8a02023780d42564c9abf55dadd929a5e83cabf620>

LOW **L-29**

ACKNOWLEDGED

mintLiquidity Causes Accounting Mismatch

CATEGORY Unexpected Behavior

LOCATION <https://github.com/GuardianOrg/HypervisorNFPM-gammapositionmanagers-team2/blob/4fa8879de218db155b108b395609fc359638c507/contracts/HypervisorNFPM.sol#L746-L772>

REVIEW Main Review

DESCRIPTION

When `HypervisorNFPM.mintLiquidity()` is invoked, funds from the contract are provided as a liquidity for a new Algebra position, however these funds are not accounted for in `getDepositAmount()` and `withdraw()`. This means that almost any call to `mintLiquidity()` will result in broken accounting - loss for current depositors and profit for the new ones if liquidity is pulled from that position in the future.

RECOMMENDATION

Consider:

1. tracking the balances of the minted positions via this function
2. pulling funds from them on `withdraw()`

RESOLUTION

Status: Acknowledged.

LOW **L-30**

RESOLVED

Zero Shares Received During Deposit

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/HypervisorNFPM-gammapositionmanagers-team1/blob/main/contracts/HypervisorNFPM.sol#L189
REVIEW	Main Review

DESCRIPTION

The `deposit` function in `HypervisorNFPM` does not verify that the calculated shares are greater than zero. This issue was previously raised and partially addressed by adding a slippage check on `deposit0` and `deposit1`. However, that check does not prevent zero-share outcomes caused by rounding errors, price manipulation, or small deposit amounts.

This can lead to situations where a user deposits tokens but receives zero shares in return—resulting in loss of funds.

Notably, the `MultiPositionManager` implementation explicitly guards against this by checking that `shares > 0`.

RECOMMENDATION

Add an explicit check:

```
require(shares > 0, "ZeroShares");
```

RESOLUTION

Status: Resolved.

<https://github.com/GammaStrategies/MultiPositionManager/commit/73f92a8fb44839bda5ae53ceb3fa1cacaf2956d3>

INFORMATIONAL I-01

RESOLVED

Incorrect Comment For removeRewardToken

CATEGORY Documentation

LOCATION <https://github.com/GuardianOrg/HypervisorNFPM-gammapositionmanagers-fuzz/blob/ca0fa1cbd65cf632f747f1f6db016c7c0ce95445/contracts/MultiFeeDistribution.sol#L166>

REVIEW Main Review

DESCRIPTION

The comment above `MultiFeeDistribution.removeRewardToken` indicates:

```
* @notice Add a new reward token to be distributed to stakers.
```

RECOMMENDATION

Update the comment to indicate that the function removes a reward token instead of adds.

RESOLUTION

Status: Resolved.

<https://github.com/GammaStrategies/HypervisorNFPM/commit/af8baa07153bdc61d434562598b43b65f2151afa#diff-cefd2887e842ad0378b8ef8a02023780d42564c9abf55dadd929a5e83cabf620>

INFORMATIONAL I-02

ACKNOWLEDGED

Incorrect Comment For Managers

CATEGORY Documentation

LOCATION <https://github.com/GuardianOrg/HypervisorNFPM-gamma/positionmanagers-fuzz/blob/988b54bb641733c5e8307cf7dd127021a5449b/contracts/MultiFeeDistribution.sol#L66>

REVIEW Main Review

DESCRIPTION

The comment for the `managers` mapping in `MultiFeeDistribution` states that the managers are "Addresses approved to call mint". However, these managers can only add or remove reward tokens.

RECOMMENDATION

Update the comment to indicate the managers' ability.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-03

RESOLVED

Missing Zero Address Check In Constructor

CATEGORY Validation

LOCATION <https://github.com/GuardianOrg/MultiPositionManagergamma-posmanager-team1/blob/main/src/MultiPositionManager.sol#L109>

REVIEW Main Review

DESCRIPTION

In the constructor, `_token0` and `_token1` are assigned to `currency0` and `currency1`. While `_token0` may be the zero address to represent native ETH, `_token1` should never be zero. Currently, there's no check to enforce this.

RECOMMENDATION

Add a validation check:

```
if (_token1 == address(0)) revert ZeroAddress();
```

RESOLUTION

Status: Resolved.

INFORMATIONAL **I-04**

RESOLVED

Use Require Over Assert

CATEGORY Best Practices

LOCATION <https://github.com/GuardianOrg/MultiPositionManagergamma-posmanager-team1/blob/main/src/MultiPositionManager.sol#L165>, <https://github.com/GuardianOrg/MultiPositionManagergamma-posmanager-team1/blob/main/src/MultiPositionManager.sol#L818>

REVIEW Main Review

— DESCRIPTION

In the `deposit` and `calcSharesAndAmounts` function, the code uses `assert(shares > 0);`, which triggers a `panic: assertion failed (0x01)` if the condition fails. This revert reason is opaque and hinders debugging.

`assert` should be reserved for testing internal errors and invariants. In production, `require` should be used instead to provide a clear and meaningful revert reason.

— RECOMMENDATION

Replace with a `require` and a clear error message:

```
require(shares > 0, "ZeroShares");
```

— RESOLUTION

Status: Resolved.

INFORMATIONAL **I-05**

ACKNOWLEDGED

Redundant Fee Initialization

CATEGORY	Informational
LOCATION	https://github.com/GuardianOrg/HypervisorNFPM-gammapositionmanagers-team1/blob/4fa8879de218db155b108b395609fc359638c507/contracts/HypervisorNFPM.sol#L44
REVIEW	Main Review

— DESCRIPTION

In HypervisorNFPM, the fee variable is initialized to 1 at declaration, and then set to 1 again in the constructor. Since both values are the same, the initial assignment is unnecessary. The same redundant pattern is present in the MultiFeeDistribution contract.

— RECOMMENDATION

Remove the redundant fee = 1 assignment and allow the constructor to handle the initialization.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-06

ACKNOWLEDGED

Liquidity May Not Fit Into uint128

CATEGORY	Informational
LOCATION	Hypervisor.sol
REVIEW	Main Review

DESCRIPTION

Whenever the Hypervisor is minting new liquidity, it should be a number fitting in `uint128`. However, it's possible to have it overflow this type, especially when the price is large. In this case, the Hypervisor can be put in a state where all of its liquidity adding functions are blocked - `deposit()` when `directDeposit()` is turned on, `rebalance()` because positions are unconditional and they spend the whole balance, etc...

RECOMMENDATION

Currently, `decreaseLiquidity()` can be used to pull liquidity out of the positions if needed, but also consider adding more granular control to `rebalance()`, i.e not using the whole balance, but part of it or having optional positions.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-07**

ACKNOWLEDGED

Outdated Comments

CATEGORY Documentation
LOCATION MultiFeeDistribution.sol
REVIEW Main Review

— DESCRIPTION

There are some outdated comments in `MultiFeeDistribution`. For example, it's stated that staked tokens cannot be withdrawn for `defaultLockDuration`, but there is no such functionality implemented.

— RECOMMENDATION

Consider refactoring the comments.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-08**

ACKNOWLEDGED

Position Manager Can Hold Its Tokens

CATEGORY Unexpected Behavior
LOCATION MultiPositionManager.sol
REVIEW Main Review

DESCRIPTION

`MultiPositionManager.deposit()` doesn't allow specifying the position manager contract itself as a recipient, but the ERC20 `transfer()` function is left unchanged which allows direct transfers to the contract. This breaks the expectation that the contract will not hold its tokens.

RECOMMENDATION

If the goal is for the contract to never hold the tokens, override the transfer function and revert if the recipient is `MultiPositionManager`.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-09**

ACKNOWLEDGED

Ambiguous Error Handling

CATEGORY	Error
LOCATION	https://github.com/GuardianOrg/MultiPositionManagerrgamma-posmanager-team2/blob/48943099b31180c466302da7b2a098e91d284405/src/MultiPositionManager.sol#L147
REVIEW	Main Review

DESCRIPTION

`MultiPositionManager.deposit()` reverts with `ZeroAddress()` error if the `to` parameter is equal to `address(0)`, but also if it's equal to `address(this)`, which can be deceiving for integrators.

RECOMMENDATION

Consider using a more general error, like `InvalidAddress()`.

RESOLUTION

Status: Acknowledged.

HIGH **H-01**

ACKNOWLEDGED

Deposit slippage protection is not sufficient

CATEGORY MEV

LOCATION <https://github.com/GuardianOrg/MultiPositionManagergamma-posmanager-team2/blob/b7667f70fbb3a2dd71d0b36ab7cac0368608ff6f/src/UniProxyV2.sol#L51-L52>

REVIEW Remediation Review

DESCRIPTION

The recently added slippage protection in `UniProxyV2` is insufficient to protect users against the issue described in the original `No Share Slippage Protection in deposit` report.

Although the check now applies the `maxSlippage` parameter to the result of the `_calculateExpectedShares()` function, the shares calculation logic is identical to `MultiPositionManager._calcSharesAndAmounts()`.

As a result, if the pool's token supplies are manipulated, the expected shares returned by `_calculateExpectedShares()` will track the manipulated state. This causes the slippage check to always pass, even when the user receives significantly fewer shares than expected under normal pool conditions.

RECOMMENDATION

Consider giving the user the ability to specify the `minShares` parameter themselves.

RESOLUTION

Status: Acknowledged.

<https://github.com/GammaStrategies/MultiPositionManager/commit/1a0d65d281c9b8d30eb51c1be9f37b71b09ed94f>

HIGH **H-02**

ACKNOWLEDGED

Whitelisted Address Bypasses Crucial Checks

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/MultiPositionManagergamma-posmanager-team1/blob/b7667f70fbb3a2dd71d0b36ab7cac0368608ff6f/src/UniProxyV2.sol#L62
REVIEW	Remediation Review

DESCRIPTION

In UniProxyV2.deposit, the clearDeposit function is skipped entirely for whitelisted addresses:

```
if (!clearance.getListed(pos, msg.sender)) {  
    clearance.clearDeposit(to, pos);  
}
```

While the intention may be to bypass unnecessary restrictions for trusted users or contracts, this also skips critical depositor protections, introducing several risks:

1. `checkTicks` is crucial to ensure the pool was not manipulated prior to deposit. Skipping this check could result in the depositor receiving far less shares than expected, losing funds in the process (fix for H-02)
2. The depositor could mistakenly deposit into a wrong or inactive position as the `onlyAddedPosition` check is skipped
3. The depositor can deposit while `ClearingV3` is paused

RECOMMENDATION

Do not skip `clearDeposit` entirely for whitelisted depositors.

Instead, follow the approach used in HypervisorNFPM, where whitelisted users still invoke `clearDeposit`, but the function internally returns early after performing only the critical safety checks.

This may be less meaningful for MultiPositionManager, but could be useful if there are future checks added to `clearDeposit` that should only apply to non-whitelisted depositors.

PROOF OF CONCEPT <https://github.com/GammaStrategies/MultiPositionManager/commit/1a0d65d281c9b8d30eb51c1be9f37b71b09ed94f>

RESOLUTION

Status: Acknowledged.

HIGH **H-03**

ACKNOWLEDGED

checkTicks Should Exclude Limit Positions

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/MultiPositionManagergamma-posmanager-team1/blob/b7667f70fbb3a2dd71d0b36ab7cac0368608ff6f/src/MultiPositionManager.sol#L359
REVIEW	Remediation Review

— DESCRIPTION

The `_checkTicks` function is intended to block deposits when the pool's current tick has been manipulated outside the vault's intended tick range. It does this by checking that the current tick lies within the lowest and highest ticks of all active positions returned by `getPositions`.

However, `getPositions` currently includes non-empty limit positions, which in some vaults are configured with extremely wide ranges (e.g., covering the entire tick range). This was observed in a production rebalance script shared by the team (`limitWidth = 10000000`).

As a result, if the min/max tick range becomes too wide, the tick check becomes effectively disabled, allowing attackers to manipulate pool price to any extreme while still passing the check.

This reopens the vault to pool manipulation attacks (e.g. H-02 style).

— RECOMMENDATION

Exclude all limit positions when calculating the tick bounds in `_checkTicks`.

Only consider base positions, which are typically narrower and represent intended liquidity deployment

PROOF OF CONCEPT <https://github.com/GammaStrategies/MultiPositionManager/commit/1a0d65d281c9b8d30eb51c1be9f37b71b09ed94f>

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-01**

ACKNOWLEDGED

Incorrect farming check

CATEGORY	DoS
LOCATION	https://github.com/GuardianOrg/HypervisorNFPM-gammapositionmanagers-team2/blob/f99d0b7d72c517de22e75a6770666577d186ce75/contracts/HypervisorNFPM.sol#L535-L537
REVIEW	Remediation Review

DESCRIPTION

A new `isEmergencyWithdrawActivated()` check has been added to `HypervisorNFPM._approveAndEnterFarming()` to not try to enter the farming if the emergency mode is activated.

```
if (incentiveId != bytes32(0) &&
    farmingCenter.eternalFarming().isIncentiveDeactivated(incentiveId) &&
    farmingCenter.eternalFarming().isEmergencyWithdrawActivated()) {
    return;
}
```

However, the check incorrectly uses `&&` which will execute the return only if all of the conditions are true. Deactivating a given incentive and activating the emergency withdrawal mode for the eternal farming are two independent actions. In result, not only the previous issue was not solved, but a new one was created - the code will call `enterFarming()` even when the incentive is deactivated.

RECOMMENDATION

You can create a separate check which returns if `isEmergencyWithdrawActivated() == true`

PROOF OF CONCEPT <https://github.com/GammaStrategies/HypervisorNFPM/commit/e9731a4032b798ebbbf1f8baafdebe7dd2e7bdd4>

RESOLUTION

Status: Acknowledged.

MEDIUM **M-02**

RESOLVED

_detectStuckRewards can block staking

CATEGORY DoS

LOCATION <https://github.com/GuardianOrg/HypervisorNFPM-gammapositionmanagers-team1/blob/f99d0b7d72c517de22e75a6770666577d186ce75/contracts/MultiFeeDistribution.sol#L414>

REVIEW Remediation Review

— DESCRIPTION

The `_detectStuckRewards` function checks whether the `HypervisorNFPM` contract has any pending rewards or bonus rewards. If rewards are detected, it returns true. This result is used in the `_stake` function of the `MultiFeeDistribution` contract to enforce a check. If `_detectStuckRewards` returns true, staking reverts.

In most cases, this should not occur because `_stake` first calls `_updateReward`, which should collect all outstanding rewards. However, the `_updateReward` function internally calls `_collectAndClaim`, which first checks for a valid incentive key before proceeding. If no valid key exists, the function returns early without collecting or claiming rewards.

There are scenarios where a position may not have a valid incentive key but still hold unclaimed rewards. For example, if `decreaseLiquidity` removes all liquidity from a position, it exits farming through `_exitFarming` without collecting or claiming rewards. Similarly, if an incentive ends or emergency withdrawal is activated and a user adds liquidity directly in Algebra, the farming state is reset and the incentive key becomes invalid. In both cases, the rewards remain unclaimed even though the position is no longer eligible.

As a result, `_detectStuckRewards` returns true, which causes the `_stake` function to revert. This prevents users from staking in the `MultiFeeDistribution` contract.

— RECOMMENDATION

Update `_collectAndClaim` to collect any remaining rewards regardless of incentive key validity, or adjust `_detectStuckRewards` to account for scenarios where rewards are no longer claimable due to expired or exited farming positions.

PROOF OF CONCEPT <https://github.com/GammaStrategies/HypervisorNFPM/commit/e9731a4032b798ebbbf1f8baafdebe7dd2e7bdd4>

— RESOLUTION

Status: Resolved.

LOW **L-01**

ACKNOWLEDGED

Zero interval TWAP still returns 0

CATEGORY Unexpected Behavior

LOCATION <https://github.com/GuardianOrg/HypervisorNFPM-gammapositionmanagers-team2/blob/f99d0b7d72c517de22e75a6770666577d186ce75/contracts/ClearingV3NFPM.sol#L202>

REVIEW Remediation Review

DESCRIPTION

The `getSqrtTwapX96()` function is still returning 0 when `_twapInterval == 0`, because the `sqrtPriceX96` variable is shadowed.

```
function getSqrtTwapX96(address pos, uint32 _twapInterval) public view returns (uint160
sqrtPriceX96) {
    if (_twapInterval == 0) {
        /// return the current price if _twapInterval == 0
        (uint160 sqrtPriceX96, , , , , ) = IHypervisor(pos).pool().safelyGetStateOfAMM();
    }
    ...
}
```

The inner `sqrtPriceX96` is a separate variable which lives only in the scope of the if statement. As a result, the function will keep returning 0.

RECOMMENDATION

Instead of creating a new variable, assign the value to the current one.

```
function getSqrtTwapX96(address pos, uint32 _twapInterval) public view returns (uint160
sqrtPriceX96) {
    if (_twapInterval == 0) {
        /// return the current price if _twapInterval == 0
        - (uint160 sqrtPriceX96, , , , , ) = IHypervisor(pos).pool().safelyGetStateOfAMM();
        + (sqrtPriceX96, , , , , ) = IHypervisor(pos).pool().safelyGetStateOfAMM();
    }
}
```

PROOF OF CONCEPT <https://github.com/GammaStrategies/HypervisorNFPM/commit/e9731a4032b798ebbbf1f8baafdebe7dd2e7bdd4#diff-a551d62ef5d903aa709b2186efce87c30b93e912fd1cc5982725026e2daf1dd3>

L-01 Zero interval TWAP still returns 0

CONTINUED

— RESOLUTION

Status: Acknowledged.

LOW **L-02**

ACKNOWLEDGED

Possible Zero Amount Transfer

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/MultiPositionManagergamma-posmanager-team1/blob/b7667f70fbb3a2dd71d0b36ab7cac0368608ff6f/src/MultiPositionManager.sol#L656
REVIEW	Remediation Review

DESCRIPTION

In `_transferIn`, zero amount transfers do not return early. This was omitted after the fix to issue L-26. As a result, if the token happens to revert on zero transfers, single-sided liquidity deposits will be blocked.

RECOMMENDATION

```
function _transferIn(address from, Currency currency, uint256 amount) internal {  
    if (currency.isAddressZero()) {  
        if (msg.value < amount) revert InvalidDepositAmount(amount);  
        if (msg.value > amount)  
            payable(msg.sender).transfer(msg.value - amount);  
    } else if (amount != 0) {  
        IERC20(Currency.unwrap(currency)).safeTransferFrom(from, address(this), amount);  
    }  
}
```

PROOF OF CONCEPT <https://github.com/GammaStrategies/MultiPositionManager/commit/1a0d65d281c9b8d30eb51c1be9f37b71b09ed94f>

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-01**

ACKNOWLEDGED

token1 can be set as address (0)

CATEGORY	Informational
LOCATION	https://github.com/GuardianOrg/MultiPositionManagergamma-posmanager-team2/blob/main/src/MultiPositionManager.sol#L120
REVIEW	Remediation Review

DESCRIPTION

The `[I-03] Missing Zero Address Check in Constructor` finding was marked as fixed, but there isn't a check added for `token1` in the constructor of the `MultiPositionManager`.

RECOMMENDATION

Consider adding the check.

RESOLUTION

Status: Acknowledged.

<https://github.com/GammaStrategies/MultiPositionManager/commit/1a0d65d281c9b8d30eb51c1be9f37b71b09ed94f>

INFORMATIONAL **I-02**

ACKNOWLEDGED

PRECISION is no longer used

CATEGORY Best Practices

LOCATION <https://github.com/GuardianOrg/HypervisorNFPM-gammapositionmanagers-team2/blob/f99d0b7d72c517de22e75a6770666577d186ce75/contracts/ClearingV3NFPM.sol#L26>

REVIEW Remediation Review

DESCRIPTION

Once the price overflow issue was fixed in `ClearingV3NFPM.sol`, the `PRECISION` constant is no longer needed.

RECOMMENDATION

Consider removing it.

PROOF OF CONCEPT <https://github.com/GammaStrategies/HypervisorNFPM/commit/e9731a4032b798ebbbf1f8baafdebe7dd2e7bdd4#diff-a551d62ef5d903aa709b2186efce87c30b93e912fd1cc5982725026e2daf1dd3>

RESOLUTION

Status: Acknowledged.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.